Homework 3: Threading and Load Tests

CS 6650 - Building Scalable Distributed Systems

Nikhil Shivakumar | Fall 2026 | October 5, 2026

Overview. This report summarizes the threading, file I/O, context-switching, and Locust load-test experiments from Homework 3. I kept the focus on the measured results, what each experiment showed, and the design changes I would make to the supplied server.

Note on the Locust numbers. The test used a 1-2 second wait_time per user and kept the total user count at 50. Because of that, adding more Locust workers was not expected to produce a large throughput increase by itself.

Part II - Thread Experiments

1. Atomicity and Race Detection

What I did: I ran many goroutines that incremented both a normal integer and an atomic integer. The expected result was 1,000,000, and I also reran the program using Go's -race flag.

Result: The normal counter finished at 156,728, while the atomic counter reached exactly 1,000,000. The race detector reported WARNING: DATA RACE for the normal counter.

What I learned: A normal increment is a read-modify-write operation, so updates can be lost when goroutines interleave. The atomic increment performs the update safely as one synchronized operation.

```
nikhil@Nikhils-MacBook-Air hw3 % go run atomic.go
Expected: 1000000
Regular Counter: 156728
Atomic Counter: 1000000
nikhil@Nikhils-MacBook-Air hw3 % go run -race atomic.go
==================
WARNING: DATA RACE
Read at 0x00c000012100 by goroutine 14:
  main.main.func1()
      /Users/nikhil/Documents/Northeastern/Sem 3/distributed-systems-course/hw3/atomic.go:26 +0x94

Previous write at 0x00c000012100 by goroutine 13:
  main.main.func1()
      /Users/nikhil/Documents/Northeastern/Sem 3/distributed-systems-course/hw3/atomic.go:26 +0xa4

Goroutine 14 (running) created at:
  main.main()
      /Users/nikhil/Documents/Northeastern/Sem 3/distributed-systems-course/hw3/atomic.go:20 +0x88

Goroutine 13 (running) created at:
  main.main()
      /Users/nikhil/Documents/Northeastern/Sem 3/distributed-systems-course/hw3/atomic.go:20 +0x88
==================
==================
WARNING: DATA RACE
Write at 0x00c000012100 by goroutine 14:
  main.main.func1()
      /Users/nikhil/Documents/Northeastern/Sem 3/distributed-systems-course/hw3/atomic.go:26 +0xa4

Previous write at 0x00c000012100 by goroutine 13:
  main.main.func1()
      /Users/nikhil/Documents/Northeastern/Sem 3/distributed-systems-course/hw3/atomic.go:26 +0xa4

Goroutine 14 (running) created at:
  main.main()
      /Users/nikhil/Documents/Northeastern/Sem 3/distributed-systems-course/hw3/atomic.go:20 +0x88

Goroutine 13 (running) created at:
  main.main()
      /Users/nikhil/Documents/Northeastern/Sem 3/distributed-systems-course/hw3/atomic.go:20 +0x88
```

Figure 1. Regular vs atomic counter and Go race-detector output.

2. Concurrent Map Experiments

Plain map baseline: The baseline uses 50 goroutines with 1,000 writes each. A normal Go map is not safe for concurrent writes and can fail with a concurrent map writes error, which is why synchronization is needed before sharing a map between writing goroutines.

Approach	Run 1	Run 2	Run 3	Mean
Mutex	14.079 ms	11.312 ms	6.796 ms	10.73 ms
RWMutex	12.575 ms	13.409 ms	7.217 ms	11.07 ms
sync.Map	11.331 ms	5.492 ms	3.136 ms	6.65 ms

Mutex: All three runs produced the correct map length of 50,000. Mutex protects the map by allowing only one goroutine at a time into the critical section.

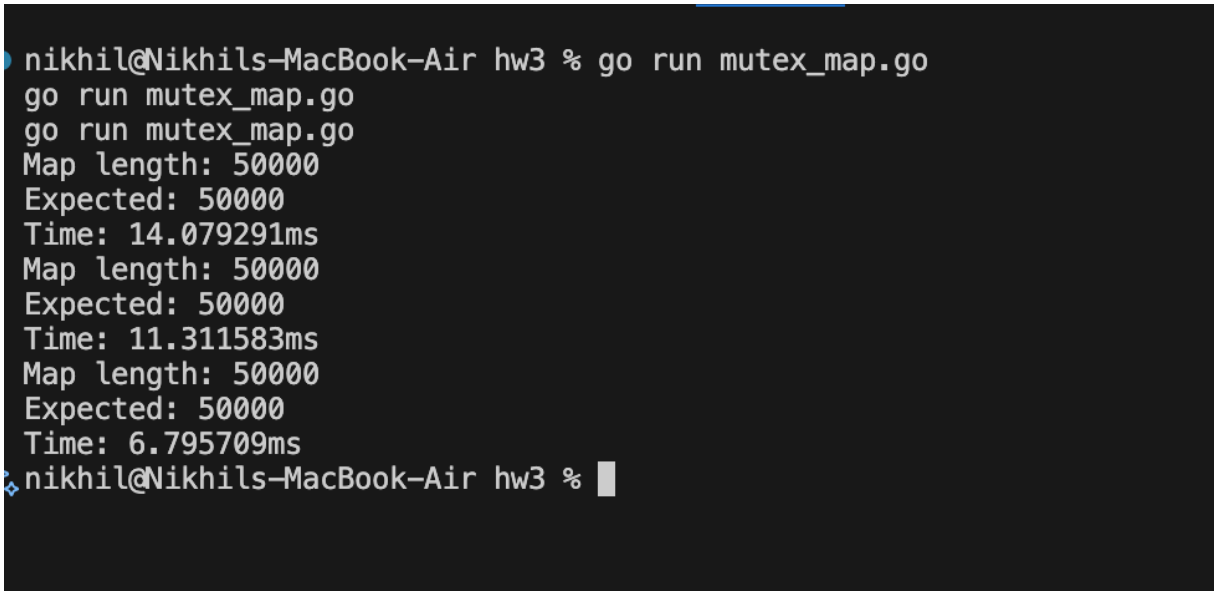


Figure 2. Mutex map results across three runs.

RWMutex: All three runs also produced 50,000 entries. It was not faster in this write-heavy test because writes still require the exclusive Lock rather than a shared read lock. RWMutex is more useful when reads dominate.



Figure 3. RWMutex map results across three runs.

sync.Map: All three runs produced 50,000 entries, with a mean time of about 6.65 ms. It was the fastest approach in these specific runs, but that does not mean it is always the best choice; the right structure depends on the access pattern.

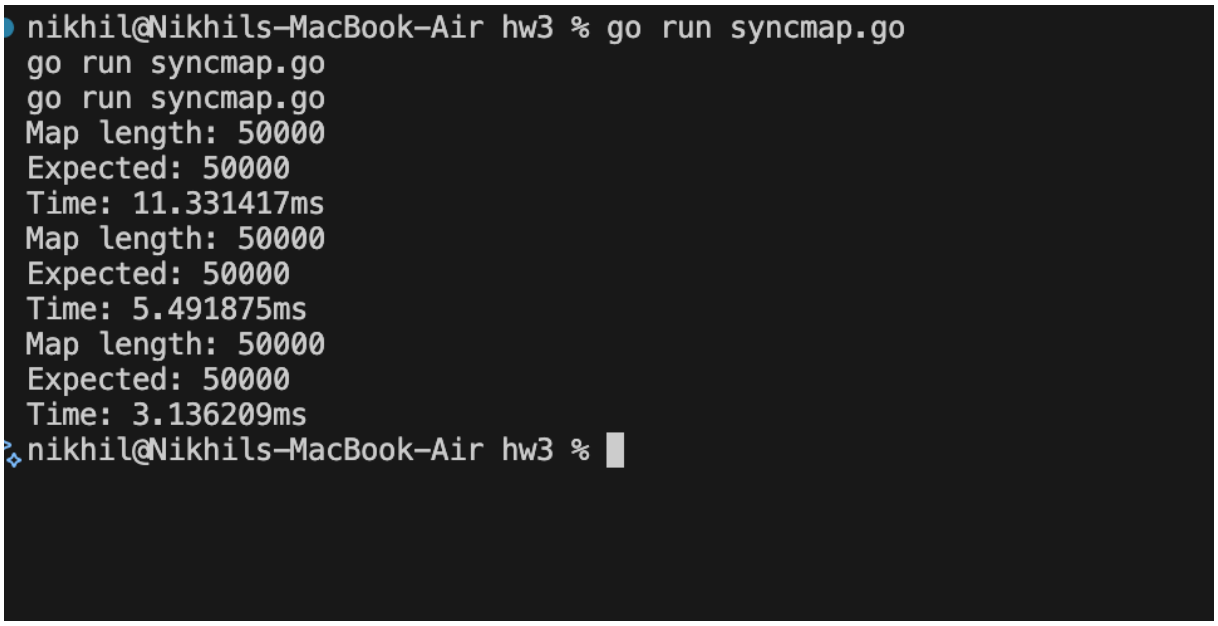


Figure 4. sync.Map results across three runs.

3. File Access

What I did: I wrote 100,000 lines directly to a file and then repeated the same work using bufio.Writer.

Result: The first set of runs averaged about 102.62 ms unbuffered and 4.79 ms buffered. Buffered output was about 21 times faster.

What I learned: Buffering combines many small writes in memory and sends them to the file in larger batches, reducing the number of expensive underlying write operations.

```
nikhil@Nikhils-MacBook-Air hw3 % cd ~/Documents/Northeastern/Sem\ 3/distributed-systems-cours
e/hw3

go run file_access.go
go run file_access.go
go run file_access.go
Unbuffered: 96.864708ms
Buffered: 4.588792ms
Unbuffered: 106.337875ms
Buffered: 5.0375ms
Unbuffered: 104.656583ms
Buffered: 4.74375ms
nikhil@Nikhils-MacBook-Air hw3 %
```

Figure 5. Buffered vs unbuffered file-writing results.

Repeat check: A second set of runs showed the same pattern: direct writes took roughly 109-132 ms, while buffered writes stayed around 4.7-5.0 ms. This confirms that the large difference was repeatable on my machine.

```
nikhil@Nikhils-MacBook-Air hw3 % go run file_access.go
go run file_access.go
go run file_access.go
Unbuffered: 132.019292ms
Buffered: 4.745667ms
Unbuffered: 120.393333ms
Buffered: 4.943375ms
Unbuffered: 109.44375ms
Buffered: 5.023542ms
nikhil@Nikhils-MacBook-Air hw3 %
```

Figure 6. Repeat file-access experiment showing the same performance gap.

4. Goroutine Context Switching

What I did: Two goroutines passed an empty signal back and forth one million times. I ran the test with GOMAXPROCS(1) and then with all 10 logical CPUs available on my machine.

Result: With GOMAXPROCS(1), the average switch times were 78 ns, 68 ns, and 69 ns (mean about 71.7 ns). With GOMAXPROCS(10), they were 90 ns, 88 ns, and 89 ns (mean 89 ns).

What I learned: For this tiny ping-pong workload, one OS thread was faster. More OS threads can add scheduler coordination and cache/synchronization overhead, so more parallelism does not automatically make every workload faster.

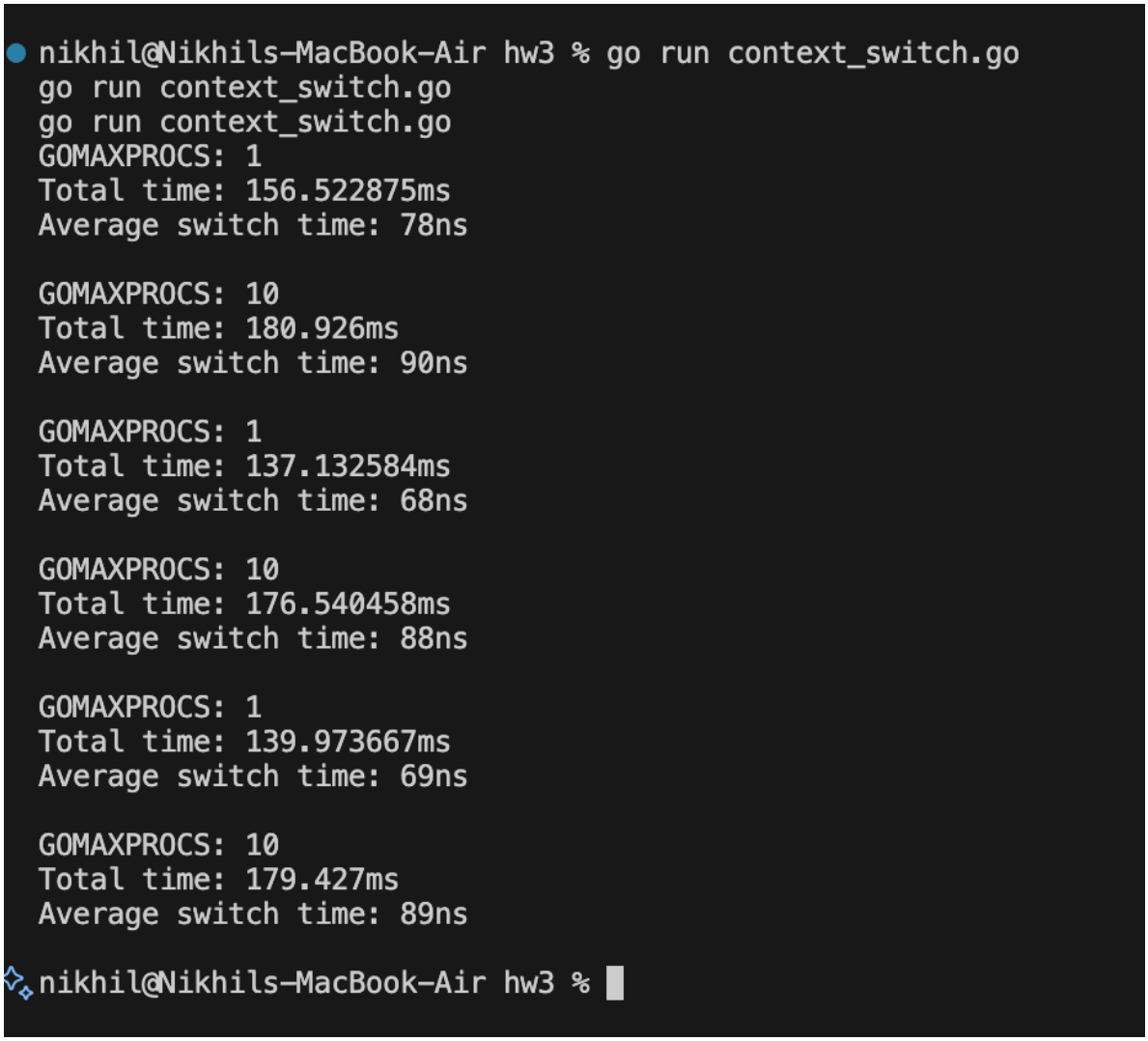
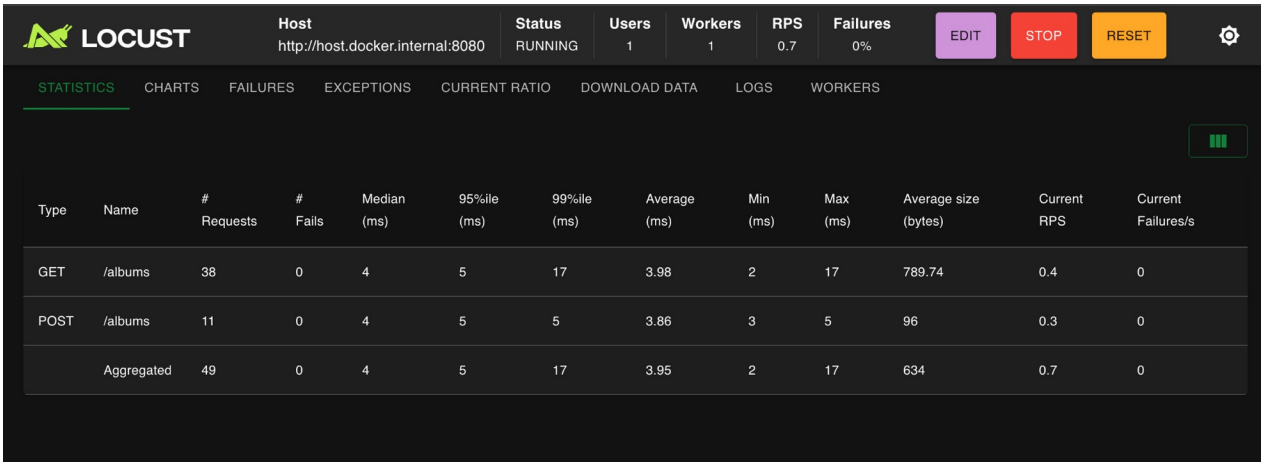


Figure 7. Context-switch results with one and ten OS threads.

Part III - Locust Load Testing

5. Baseline: 1 User, 1 Worker

Result: GET and POST requests both completed with zero failures. The aggregate request rate was about 0.7 RPS, which is reasonable with one user waiting 1-2 seconds between tasks.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	38	0	4	5	17	3.98	2	17	789.74	0.4	0
POST	/albums	11	0	4	5	5	3.86	3	5	96	0.3	0
	Aggregated	49	0	4	5	17	3.95	2	17	634	0.7	0

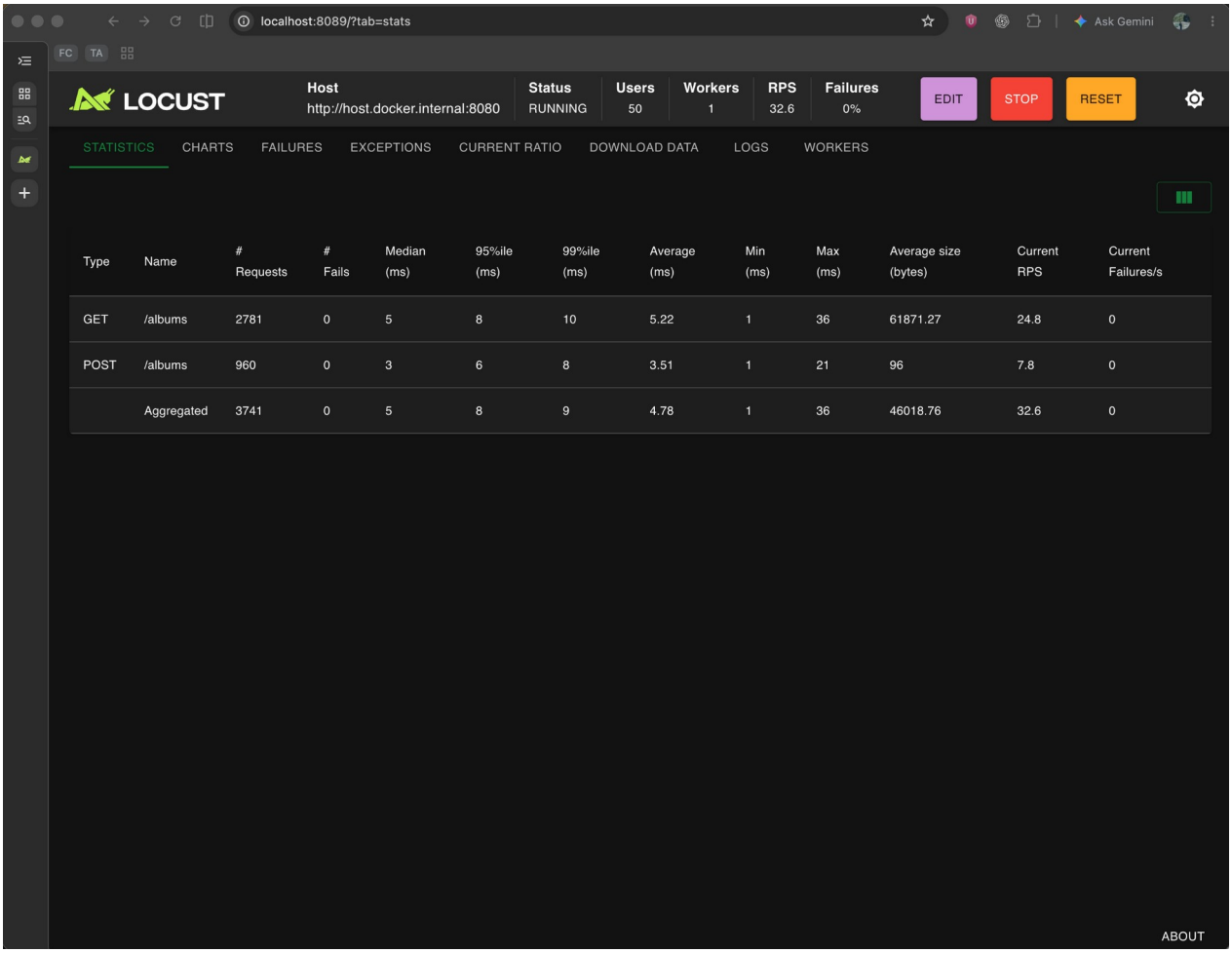
Figure 8. Locust baseline with 1 user and 1 worker.

6. 50 Users, 1 Worker

Configuration: 50 users, 10 users/second ramp-up, one Locust worker, and a 3:1 GET-to-POST task ratio.

Result: Aggregate throughput was 32.6 RPS with zero failures. GET median/p95/p99 were 5/8/10 ms. POST median/p95/p99 were 3/6/8 ms.

Interpretation: The server remained stable at this load. GET returned a much larger response body than POST, but both request types still had low latency in this test.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	2781	0	5	8	10	5.22	1	36	61871.27	24.8	0
POST	/albums	960	0	3	6	8	3.51	1	21	96	7.8	0
	Aggregated	3741	0	5	8	9	4.78	1	36	46018.76	32.6	0

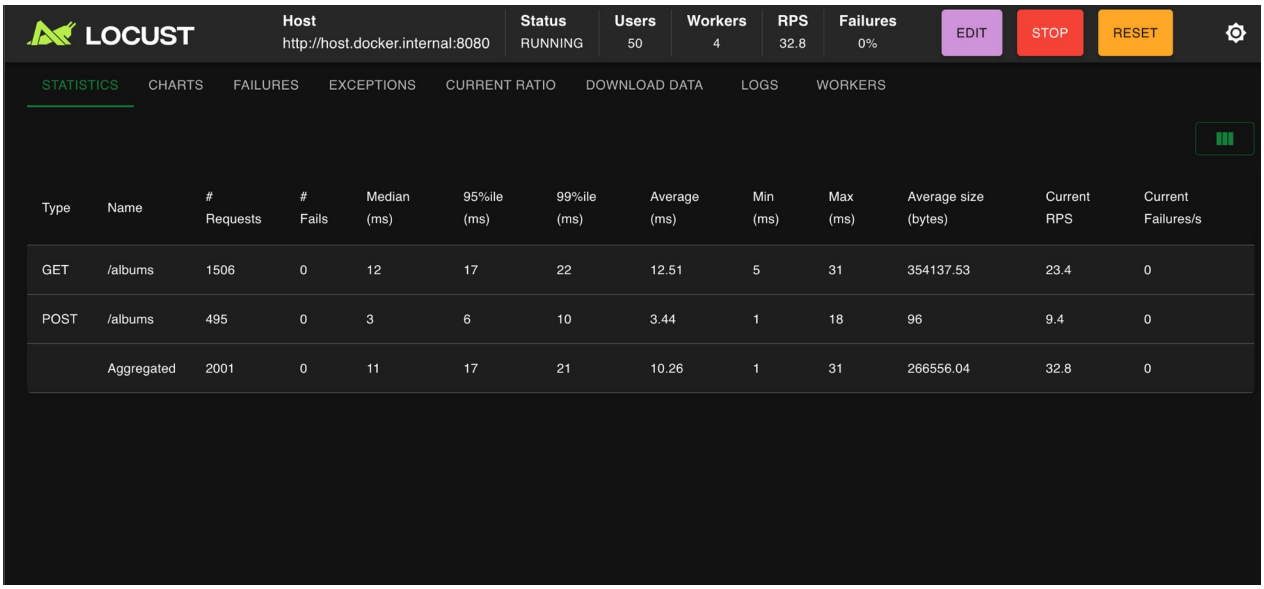
Figure 9. Locust statistics for 50 users and 1 worker.



Figure 10. Throughput, response-time, and user-count charts.

7. Four Workers and FastHttpUser

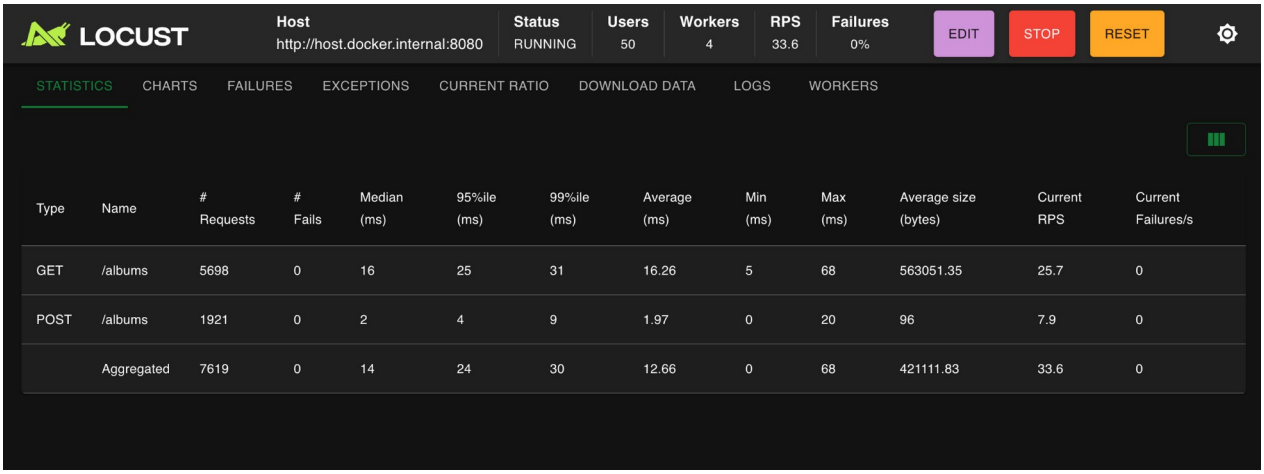
Four workers: With the same 50 total users, throughput was 32.8 RPS with zero failures. This is almost the same as the one-worker result. The reason is that the workload was still limited by the same total user count, per-user wait time, and server-side work. Adding load-generator workers alone does not remove those limits.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	1506	0	12	17	22	12.51	5	31	354137.53	23.4	0
POST	/albums	495	0	3	6	10	3.44	1	18	96	9.4	0
	Aggregated	2001	0	11	17	21	10.26	1	31	266556.04	32.8	0

Figure 11. Locust statistics for 50 users distributed across 4 workers.

FastHttpUser: FastHttpUser produced 33.6 RPS with zero failures. The increase was small because the normal Locust HTTP client was not the main bottleneck at this load level.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	5698	0	16	25	31	16.26	5	68	563051.35	25.7	0
POST	/albums	1921	0	2	4	9	1.97	0	20	96	7.9	0
	Aggregated	7619	0	14	24	30	12.66	0	68	421111.83	33.6	0

Figure 12. FastHttpUser result with 50 users and 4 workers.

Part IV - Concurrent Server Design

Current limitation: The supplied server accepts one connection and then calls Handle_client(conn) directly. In the shown path, another client cannot be accepted until that call returns, so a slow or long-lived client can delay everyone else.

```
// Accept the first connection
conn, err := ln.Accept()
if err != nil {
    fmt.Println("Error in accepting")
}

Handle_client(conn)
```

Figure 13. Current server path: Accept() followed by a synchronous Handle_client(conn).

Proposed Design

- Put Accept() inside a loop so the server continues accepting new clients.
- Start a goroutine for each accepted connection: go Handle_client(conn). This lets independent clients make progress at the same time.
- Keep the Album data type unchanged, but place shared storage and ID generation behind synchronization. Use sync.RWMutex around the shared store.
- Use RLock/RUnlock for read-only commands such as GET, SEARCH, and DUMP. Use Lock/Unlock for STORE, and keep the entire check-for-existing + ID-allocation + insert sequence inside the same write lock so it is atomic.
- For SEARCH and DUMP, copy/snapshot the matching data while the read lock is held, release the lock, and only then write the response to the network. A slow client should not keep the shared store locked while bytes are being sent.
- Bound resource use. A semaphore can cap active request processing; if stronger fairness is needed, parsed requests can be placed in a bounded FIFO work queue handled by a fixed worker pool.

Core accept loop (conceptual):

```
for {
    conn, err := ln.Accept()
    if err != nil {
        continue
    }
    go Handle_client(conn)
}
```

How the Design Scales

Response rate: Throughput should improve as independent clients overlap, especially when some requests spend time waiting on network I/O. The increase will not be perfectly linear because requests still share CPU, memory, storage, and locks. This is the same basic limit described by Amdahl's Law.

Starvation: A goroutine per connection prevents one client from blocking the accept loop, but it does not guarantee perfect fairness by itself. Keeping critical sections short and bounding active work prevents resource exhaustion. If strict request fairness becomes important, a FIFO request queue with a fixed worker pool is a clearer solution.

Worst case: A write-heavy workload can serialize on the write lock. Large SEARCH or DUMP operations can also become expensive as the collection grows. I would reduce lock duration, snapshot results before network writes, and,

if the dataset grows substantially, consider sharding the store and adding an index for title search instead of scanning every album.

Data structures: The Album structure does not have to change. The storage wrapper may change to hold the map/indexes, the RWMutex, and the next-ID state together so all shared mutable state is protected consistently.

Overall Takeaways

- Shared mutable state needs synchronization; otherwise races or runtime failures can occur.
- Mutex, RWMutex, and sync.Map have different tradeoffs. RWMutex is most useful when reads dominate, while write-heavy workloads still serialize.
- Buffered file I/O was dramatically faster than repeated small direct writes in my measurements.
- More threads or load-generator workers do not automatically increase throughput; the actual bottleneck and workload shape matter.
- A concurrent server should separate client handling, protect shared state, keep lock durations short, and bound resource use.

Repository note: The experiment code should also be committed to the course repository, as required by the assignment.